Semantic Overhead

Addressing Wasted Compute in Agentic Coding

Toward Constitutional Compression of Meaning in Software Systems

---

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Modern software development–particularly in agentic and AI-assisted environments–incurs substantial hidden computational cost due to fragmented, ambiguous, and repeatedly reconstructed semantics. This paper introduces the concept of **semantic overhead**: the portion of compute expenditure devoted not to producing or executing systems, but to recovering, reconciling, and reinterpreting their intended meaning.

We argue that contemporary pipelines distribute semantics across code, documentation, prompts, tickets, and human memory, forcing both humans and machines into continuous cycles of inference, validation, and repair. This results in systemic inefficiency, characterized by redundant model calls, iterative regeneration, and coordination drift.

We propose **Constitutional Agentic Compilation (CAC)** as a structural remedy. By concentrating system semantics into a governed pipeline–**PCM -> CONDEX -> HVL -> Codex -> MCIR -> PEIR**–CAC reduces ambiguity, bounds reasoning, and enables deterministic or bounded-discretion realization. This transformation shifts software production from high-entropy, interpretation-heavy processes toward lower-entropy, constitutionally governed systems.

We formalize semantic overhead as a measurable inefficiency class, introduce the concepts of **semantic debt**, **semantic locality**, and the **Semantic Thrash Threshold**, and analyze how constitutional specification compresses coordination cost across development, maintenance, and organizational scales. The implications extend beyond software engineering to data center efficiency, enterprise coordination, environmental impact, and long-horizon AI system stability.

---

1. The Problem of Semantic Overhead

In contemporary software systems, meaning is not localized. It is distributed across:

- source code
- documentation
- prompts and agent instructions
- issue trackers and tickets
- architectural diagrams
- implicit human knowledge

This distribution forces every participant–human or machine–to repeatedly reconstruct intent.

We define:

> **Definition (Semantic Overhead).** Semantic overhead is the fraction of computational and human effort spent recovering, reconstructing, validating, or reconciling system meaning rather than executing or realizing it.

This overhead manifests as:

- repeated prompting and regeneration
- validation cycles and test failures
- patching and rework
- cross-team misalignment
- drift between specification and implementation

In agentic systems, this cost is amplified. Each model invocation operates under partial context, requiring inference to fill gaps. The result is not just inefficiency, but systematic redundancy.

At a deeper level, semantic overhead is often a **search problem over latent meaning space**.

Agents search for intent.

Validation prunes the search.

Retries expand it again.

This paper argues that semantic overhead is the cost of unconstrained or weakly constrained search over latent semantic space.

---

2. Formalizing Semantic Overhead

Semantic overhead may be modeled as:

$$SO = \frac{C_r + C_i + C_v + C_d}{C_t}$$

Where:

- (C_r) = reconstruction cost
- (C_i) = inference cost
- (C_v) = validation / reconciliation cost
- (C_d) = drift correction cost
- (C_t) = total compute or labor cost

Thus:

$$Useful\ Compute\ Ratio = 1 - SO$$

We may also define:

> **Semantic Compression Ratio (SCR)**
>
> $$SCR = \frac{SO_{before}}{SO_{after}}$$
>
> This provides a measurable indicator of constitutional compression effectiveness.

Semantic overhead exists in both machine and human form:

$$SO_{total} = SO_{machine} + SO_{human}$$

Where human semantic overhead includes:

- meetings
- onboarding
- clarification loops
- architecture rediscovery
- organizational coordination

---

3. Sources of Wasted Compute

Semantic overhead arises from five primary conditions:

3.1 Fragmented Meaning

No single artifact contains authoritative semantics. Instead:

- code encodes behavior
- docs describe intent
- prompts guide execution
- humans reconcile discrepancies

Each layer is incomplete. Alignment must be re-derived continuously.

3.2 Repeated Inference

Agents repeatedly infer:

- system structure
- constraints
- invariants
- intent behind code

This produces cycles of:

- generation
- evaluation
- correction

These cycles consume compute without advancing the system's semantic state.

3.3 Drift and Reconciliation

Over time:

- implementations diverge from intent
- documentation becomes stale
- teams develop inconsistent interpretations

This creates reconciliation work:

- audits
- refactoring
- revalidation

Drift converts prior compute investment into ongoing cost.

3.4 Cold-Start Penalty

New agents or developers must:

- reconstruct system context
- infer architecture
- rediscover constraints

This imposes repeated onboarding cost at every entry point.

Cold-start cost is effectively a **full semantic cache miss**.

Each new participant rebuilds the same semantic model, pays the same inference cost, and introduces variance.

CAC introduces a shared semantic cache:

the Codex.

This shifts retrieval closer to O(1) lookup rather than O(n) reconstruction.

3.5 Low Semantic Locality

We define:

> **Semantic Locality**: the degree to which system meaning can be resolved from a bounded, authoritative artifact without external reconstruction.

Traditional systems exhibit low semantic locality.

Meaning is spread across artifacts and institutional memory.

CAC increases semantic locality by concentrating meaning into governed constitutional artifacts.

---

4. Semantic Debt

Semantic overhead is the current recurring cost of ambiguity.

Semantic debt is the accumulated unresolved ambiguity likely to increase future semantic overhead.

Examples include:

- undocumented assumptions
- stale documentation
- implicit architectural knowledge
- conflicting prompts or instructions

Like technical debt, semantic debt compounds over time.

Semantic debt is future overhead in latent form.

---

5. Semantic Thrash and System Instability

Semantic overhead compounds over time, producing what can be described as **semantic thrash**:

> A regime in which meaning is no longer stable across interactions, requiring constant re-derivation.

We define:

> **Semantic Thrash Threshold (STT)** The point at which the marginal compute cost of maintaining semantic consistency exceeds the marginal compute value of new realization.

Operationally:

$$\frac{dC_{overhead}}{dt} > \frac{dC_{useful}}{dt}$$

Observable signals include:

- retries per task increasing
- divergence between outputs and intent widening
- validation cycles growing faster than feature output

This aligns with systems that remain locally coherent but globally inconsistent.

---

6. Constitutional Compression of Meaning

CAC addresses semantic overhead by introducing a governed semantic pipeline:

```
text id="4xdy7u" PCM v CONDEX v HVL v Codex v MCIR v PEIR
```

This produces:

> **Semantic compression** – the reduction of distributed meaning into governed constitutional form.

More precisely:

CAC converts **semantic reconstruction** into **semantic reuse**.

Traditional systems recompute meaning.

CAC persists and references prior crystallization.

6.1 PCM – Pre-Crystallized Material

Raw semantic matter:

- transcripts
- tickets
- notes
- documents
- chats

6.2 CONDEX – Context Index

Indexed, authority-tagged evidence extracted from PCM.

6.3 HVL – Human Validation Layer

Human-governed coherence review before crystallization.

6.4 Codex

Explicit invariants, configuration space, and realization scaffolds.

6.5 MCIR

Canonical machine-legible meaning representation.

6.6 PEIR

Executable realization.

---

7. The Codex as Semantic Concentration

The Constitutional Codex:

- encodes invariants (what cannot change)
- defines configuration space (what may vary)
- embeds reasoning scaffolds
- declares verification surfaces

This eliminates the need to reconstruct meaning from multiple sources.

---

8. Reduction of Interpretation Cycles

With constitutional compression:

- agents do not infer system structure–they read it
- constraints are explicit, not latent
- ambiguity is bounded, not explored

This reduces:

- regeneration loops
- failed attempts
- corrective cycles

Semantic overhead exists because systems repeatedly re-perform constitutional binding from scratch.

CAC preserves prior crystallization.

---

9. Compute Implications

Semantic overhead is not just a cognitive cost–it is a **compute cost**.

In agentic pipelines, overhead manifests as:

- excess model invocations
- repeated token consumption
- retries and backtracking
- validation reruns

CAC reduces these costs by:

- constraining the search space
- eliminating redundant inference
- enabling first-pass correctness under bounds

9.1 From Exploration to Execution

Current paradigm:

> generate broadly -> validate late -> repair repeatedly

CAC paradigm:

> specify tightly -> realize within bounds -> verify directly

This shifts compute from exploratory search to bounded execution.

9.2 Example Case

A useful comparison is not CAC versus no specification at all.

A fairer comparison is:

CAC versus conventional specification.

A thorough design document or implementation spec already reduces semantic overhead significantly.

CAC's novelty is the **constitutional delta**:

the additional reduction achieved through:

- invariant / configuration separation
- admissibility gates
- governed amendment pathways
- explicit verification surfaces

Illustrative expected-value comparison:

No specification:

- 8 prompts
- 3 failed tests
- 2 regeneration loops
- 1 architectural misunderstanding

Conventional detailed specification:

- 3 prompts
- 1 failed test
- occasional regeneration

CAC workflow:

- 1 Codex load
- 1 realization pass
- 1 verification cycle

The claim is not that specification itself is novel.

The claim is that constitutional specification further reduces branching factor, lowers expected retries, and preserves reusable semantic crystallization.

9.3 Data Center and Environmental Impact

At scale, reduced semantic overhead implies:

- fewer model calls
- reduced token throughput
- lower GPU time
- reduced energy consumption
- improved throughput

The causal chain is direct:

Semantic overhead -> increased token throughput -> increased GPU time -> increased energy consumption

Semantic inefficiency is environmental inefficiency.

Wasted meaning is wasted electricity.

Even modest reductions in thrash produce significant infrastructure savings.

---

10. Organizational Coordination Effects

Semantic overhead is not confined to machines. It dominates human coordination cost.

Organizations spend substantial effort on:

- aligning teams on intent
- maintaining consistency across artifacts
- auditing systems for correctness
- resolving mismatches between design and implementation

CAC reduces this by:

- centralizing semantics
- making invariants explicit
- enabling direct verification

10.1 Systems as Living Constitutions

Under CAC, systems resemble **living constitutions**:

- governing structure is explicit
- changes are amendments, not rewrites
- invariants persist across versions

This produces:

- lower regression risk
- clearer auditability
- reduced coordination overhead

10.2 Reduced Cold-Start Cost

New agents or developers:

- load the Codex
- immediately access system semantics
- do not reconstruct context from fragments

This enables:

- faster onboarding
- consistent reasoning across participants
- reduced duplication of effort

---

11. Maintenance and Evolution

The long-term impact of CAC is most pronounced in maintenance.

Traditional systems accumulate:

- semantic entropy
- undocumented assumptions
- divergence between layers

CAC mitigates this by:

- preserving semantics at the constitutional layer
- enabling regeneration of implementation
- decoupling meaning from code

11.1 Structural Updates

Changes become:

- targeted modifications to invariants or configuration
- propagated through realization

Instead of:

- broad code rewrites
- manual synchronization across artifacts

11.2 Reduced Entropy Growth

Because meaning is centralized:

- drift is detectable
- inconsistencies are localized
- updates do not require rediscovery of intent

---

12. Beyond Software: Generalized Coordination Compression

Semantic overhead is a general phenomenon wherever:

- multiple agents coordinate
- meaning is distributed
- interpretation is required

CAC suggests a broader pattern:

> **centralize semantics -> reduce coordination cost**

This applies to:

- enterprise workflows
- research pipelines
- government systems
- multi-agent AI environments

---

13. Rebound Effects

Reducing semantic overhead may increase system ambition.

Organizations may use saved compute and coordination capacity to build:

- larger systems
- more complex workflows
- more ambitious automation

This resembles Jevons paradox.

Efficiency gains may increase total system complexity.

This does not negate the thesis.

It reinforces the need for constitutional structure at scale.

---

14. Limitations and Boundary Conditions

CAC does not eliminate all overhead.

Residual costs remain in:

- initial Codex authoring
- invariant correctness
- domain modeling accuracy

Incorrect specifications produce:

- systematically wrong systems
- though still internally consistent

There is also a dual failure mode:

over-compression.

If the Codex is:

- too rigid
- incorrectly abstracted
- prematurely constraining

then the system may become efficiently wrong.

Semantic thrash is too much entropy.

Over-compression is too little flexibility.

CAC must balance both.

---

15. Conclusion

Semantic overhead represents a major, under-recognized source of inefficiency in modern software and AI systems. It arises from the fragmentation and continual reconstruction of meaning across artifacts, agents, and time.

Constitutional Agentic Compilation addresses this by:

- compressing semantics into a governed pipeline
- bounding interpretation
- enabling efficient realization

The result is a shift from:

> high-entropy, interpretation-heavy systems

to:

> lower-entropy, constitutionally governed systems

This transformation has implications not only for developer productivity, but for:

- compute efficiency
- data center utilization
- environmental sustainability
- organizational coordination
- long-horizon system stability

Constitutional systems do not merely reduce code generation cost;

they reduce the **computational thermodynamics of ambiguity**.

Standing and Claim Boundary

This paper is a conceptual, formal, or architectural contribution unless it explicitly binds a statement to a cited external source or an exact carried evidence artifact. Unbound statements about implementation, validation, performance, safety, universality, or proof are research claims or design obligations, not established operational facts. Corpus-internal references establish conceptual lineage and comparison, not independent external confirmation.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Possibility Space.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.
- **Revision.** The present paper does not claim a complete revision treatment. That stage remains successor work and cannot be inferred from the architecture described here.

Corpus-Wide Reference Positioning

This paper argues that modern software development–particularly in agentic and AI-assisted environments–incurs substantial hidden computational cost due to fragmented, ambiguous, and repeatedly reconstructed semantics. Its nearest external comparison fields are semantic, software, governance. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Richard Cyganiak, David Wood, and Markus Lanthaler, eds. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 25 February 2014. https://www.w3.org/TR/rdf11-concepts/ **Relationship:** Adjacent graph data model and identity baseline.

[**E2**] Alistair Miles and Sean Bechhofer, eds. *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation, 18 August 2009. https://www.w3.org/TR/skos-reference **Relationship:** Adjacent standard for concepts, labels, schemes, notes, and semantic mappings.

[**E3**] W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview*, Second Edition. W3C Recommendation, 11 December 2012. https://www.w3.org/TR/owl2-overview/ **Relationship:** Adjacent formal ontology standard used to compare declared semantics and inference commitments.

[**E4**] Mark D. Wilkinson et al. "The FAIR Guiding Principles for Scientific Data Management and Stewardship." *Scientific Data* 3, 2016, 160018. DOI: 10.1038/sdata.2016.18. https://doi.org/10.1038/sdata.2016.18 **Relationship:** Predecessor stewardship principles for findability, accessibility, interoperability, and reuse.

[**E5**] Peter Gardenfors. *Conceptual Spaces: The Geometry of Thought*. MIT Press, 2000. mitpress.mit.edu source **Relationship:** Predecessor geometric treatment of concepts; cited as comparison, not as validation of the paper's manifold terminology.

[**E6**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E7**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715.

https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

Ableman corpus

[**A1**] Adam Ableman. *Meaning as Binding.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-5F976917F0B283EA`;
source SHA-256
`A470D1A0B9ED7C3CEAB6BB88BE4B642A2BA36427447F787764329FD574DA0C97`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Governed Meaning Base (GMB).* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D95CCC4FB7B58BFB`;
source SHA-256
`A3B62E50F8BEE125FDAE0D38C2839AFD1CCC138296268BF93217481E38AA48EA`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Semantic Manifold Topology.* Ableman Corpus current edition; work identity `PAPER.PUB.20257244`;
source SHA-256
`B8F28CC652A73A3EE4CF6FD0E23C1ABC922A6660319B2FB4CC1CCAAA606EC64A`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Global Term Key: Identity Primitives in Governed Knowledge Systems.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-358AC09F5A13587E`;
source SHA-256
`AFB29CFCB4B58A78B79709AD453CEE9088CCB2AE40F30A70260F1B54F5160579`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Semantic Legitimacy Before Context.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-091E34BA93992CE4`;
source SHA-256
`DEB7626CE1AB54F4B541347DCC3C4E14DC4F682230ACCCE1A93D1D536A50C44C`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Sovereign Software.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;
source SHA-256

6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems.* Ableman Corpus current edition; work identity PAPER.PUB.20472196;
source SHA-256
3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *Semantic Crystallization.* Zenodo, 2026. doi:10.5281/zenodo.20310115

Adam Ableman *Semantic Debt.* Zenodo, 2026. doi:10.5281/zenodo.21111666

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848770